

Master-Blueprint: Gitter-WhatsApp-Bridge (Bidirektional)

Dieses Dokument fasst das Gesamtsystem, die Architektur sowie alle operativen Schritte aus Anwender- und Entwicklerperspektive zusammen.

1. Systemübersicht & Architektur

Die Brücke verbindet einen Gitter-Raum (Matrix/IRC-nah für Entwickler-Communities) mit einer WhatsApp-Gruppe für schnellen mobilen Zugriff und Interaktion.



- **Richtung A (Gitter → WhatsApp):** Neue Chat-Events im Gitter-Stream werden geparkt, Markdown wird WhatsApp-kompatibel formatiert und an die hinterlegte WhatsApp-Gruppen-ID gepusht.
- **Richtung B (WhatsApp → Gitter):** Eingehende Nachrichten in der WhatsApp-Gruppe werden validiert (keine Bots, korrekte Absender) und via Gitter REST-API in den Raum gepostet.

2. Rollen & Aufgaben (Wer muss was tun?)

A. Anwenderseite (Nutzer auf WhatsApp-Ebene)

1. **Gruppe beitreten:** Der Nutzer tritt der dedizierten WhatsApp-Gruppe bei (z. B. via Einladungslink).
2. **Benachrichtigungseinstellungen:** Empfohlen: Eigene Benachrichtigungstöne setzen oder bei hohem Kanalrauschen die Gruppe stummschalten und Mentions nutzen.
3. **Bedienung:**
 1. **Antworten:** Einfach Text in die Gruppe tippen – die Bridge spiegelt ihn mit dem WhatsApp-Profilnamen in Gitter.
 2. **Befehle (falls implementiert):** Steuerbefehle wie `!status`, `!users` oder `!mute` können direkt im Chat abgesetzt werden.

B. Entwicklungs- / Betreiberseite (Host & Middleware)

1. **Host-Umgebung bereitstellen:** Ein Rechner oder Server (z. B. VPS, Heimserver oder Raspberry Pi), der rund um die Uhr läuft.
2. **WhatsApp-Session aufsetzen:**
 1. Empfehlung: Eigene Prepaid-SIM / Zweitnummer für den Bot-Account verwenden (Schutz der privaten Hauptnummer vor Sperren).
 2. Node.js-Dienst mit `LocalAuth` (persistente Session-Tokens) starten und den QR-Code über WhatsApp unter *Verknüpfte Geräte* einscannen.
3. **Gruppen-ID ermitteln:** Einmalig eine Testnachricht in die WhatsApp-Gruppe schreiben, ID (Format `xxxxxxx@g.us`) aus den Konsolen-Logs ablesen und in `.env` hinterlegen.
4. **Schleifenerkennung absichern:** Bot-eigene IDs filtern, damit gespiegelte Nachrichten nicht endlos im Kreis gesendet werden.
5. **Prozessüberwachung:** Dienst via `pm2` oder `systemd` absichern, damit er nach Netzwerkabbrüchen oder Neustarts automatisch wieder anläuft.

3. Gitter & GitHub: Implikationen & Setup

Bereich	Anforderung / Implikation	Maßnahme
Gitter-Authentifizierung	API-Zugriff benötigt persönlichen Token	Ein Gitter-Konto (über GitHub oder Matrix) registrieren und auf <code>developer.gitter.im</code> den Personal Access Token generieren.
Gitter-Raum-ID	Zielraum muss eindeutig adressiert werden	Über <code>GET</code> <code>[https://api.gitter.im/v1/rooms]</code> <code>(https://api.gitter.im/v1/rooms)</code> die hexadezimale <code>roomId</code> des Zielkanals ermitteln.
GitHub-Verknüpfung	Gitter-Räume sind oft an GitHub-Repos gekoppelt	Wenn der Gitter-Raum an ein GitHub-Repo gebunden ist, generieren Aktivitäten (PRs, Issues, Commits) standardmäßig Chat-Events. Die Bridge sollte filtern, ob

Bereich	Anforderung / Implikation	Maßnahme
		Repo-Aktivitäts-Nachrichten ebenfalls an WhatsApp weitergeleitet werden sollen (Rauschunterdrückung).
GitHub-Tokens	Keine direkten GitHub-Tokens für reines Chatten nötig	Die Gitter-API kapselt den Chat vollständig ab. GitHub-Tokens werden nur gebraucht, wenn der Bot parallel Issues oder PR-Kommentare direkt auf GitHub manipulieren soll.

...

4. Technische Implementierung (Schritt-für-Schritt)

Schritt 1: Projekt-Initialisierung

Bash



```
mkdir gitter-whatsapp-bridge
cd gitter-whatsapp-bridge
npm init -y
npm install whatsapp-web.js qrcode-terminal axios dotenv
```

Schritt 2: Konfiguration (`.env`)

Ini, TOML



```
GITTER_TOKEN=dein_gitter_personal_access_token
GITTER_ROOM_ID=deine_gitter_room_id
WHATSAPP_GROUP_ID=123456789-987654@g.us
GITTER_BOT_USER_ID=deine_eigene_gitter_user_id
```

Schritt 3: Brücken-Skript (`bridge.js`)

```
require('dotenv').config();
const { Client, LocalAuth } = require('whatsapp-web.js');
const qrcode = require('qrcode-terminal');
const axios = require('axios');

const GITTER_API_BASE = 'https://api.gitter.im/v1';
const GITTER_STREAM_BASE = 'https://stream.gitter.im/v1';

// 1. WhatsApp-Client initialisieren
const waClient = new Client({
  authStrategy: new LocalAuth(),
  puppeteer: {
    args: ['--no-sandbox', '--disable-setuid-sandbox']
  }
});

waClient.on('qr', (qr) => {
  console.log('[WhatsApp] QR-Code scannen:');
  qrcode.generate(qr, { small: true });
});

waClient.on('ready', () => {
  console.log('[WhatsApp] Bot ist online und verbunden.');
```

```
  startGitterStream();
});

// 2. WhatsApp -> Gitter
waClient.on('message', async (msg) => {
  // Nur Nachrichten aus der definierten Gruppe weiterleiten
  if (msg.from !== process.env.WHATSAPP_GROUP_ID) {
    // Loggt fremde/neue IDs zum einfachen Auslesen
    console.log(`[WhatsApp-Log] Nachricht von ${msg.from}: ${msg.body}`);
    return;
  }

  // Eigene Bot-Nachrichten ignorieren
  if (msg.fromMe) return;

  const contact = await msg.getContact();
  const senderName = contact.pushname || contact.number || 'WhatsApp User';
  const payload = {
    text: `**[WhatsApp] ${senderName}:** ${msg.body}`
  };

  try {
    await axios.post(
```

```

        `${GITTER_API_BASE}/rooms/${process.env.GITTER_ROOM_ID}/chatMessages`,
        payload,
        { headers: { Authorization: `Bearer ${process.env.GITTER_TOKEN}` } }
    );
} catch (err) {
    console.error('[Gitter-Send-Error]:', err.response?.data || err.message);
}
});

// 3. Gitter -> WhatsApp (Streaming)
async function startGitterStream() {
    try {
        const streamUrl = `${GITTER_STREAM_BASE}/rooms/${process.env.GITTER_ROOM_ID}/`;
        const response = await axios({
            method: 'get',
            url: streamUrl,
            headers: { Authorization: `Bearer ${process.env.GITTER_TOKEN}` },
            responseType: 'stream'
        });

        console.log('[Gitter] Stream erfolgreich verbunden.');
- let buffer = '';
- response.data.on('data', (chunk) => {
            buffer += chunk.toString();
            const lines = buffer.split('\n');
            buffer = lines.pop(); // Unvollständige Zeile im Buffer behalten

            for (const line of lines) {
                const trimmed = line.trim();
                if (!trimmed) continue;

                try {
                    const message = JSON.parse(trimmed);

                    // Schleifenerkennung: Eigene Nachrichten ignorieren
                    if (message.fromUser?.id === process.env.GITTER_BOT_USER_ID) return;
                    if (message.text.startsWith('*[WhatsApp]')) return;

                    // Formatierung & Weiterleitung an WhatsApp
                    const author = message.fromUser?.displayName || message.fromUser?
                    const waText = `[Gitter] ${author}:*\n${formatMarkdownForWhatsApp(
                        message.text
                    )}`;

                    waClient.sendMessage(process.env.WHATSAPP_GROUP_ID, waText);
                } catch (e) {
                    console.error('[Stream-Parse-Error]:', e.message);
                }
            }
        });
    }
}

```

```

    response.data.on('end', () => {
      console.warn('[Gitter] Stream beendet. Reconnect in 5 Sekunden...');
      setTimeout(startGitterStream, 5000);
    });

  } catch (err) {
    console.error('[Gitter-Stream-Error]:', err.message);
    setTimeout(startGitterStream, 5000);
  }
}

// Hilfsfunktion: Markdown grob an WhatsApp-Syntax anpassen
function formatMarkdownForWhatsApp(text) {
  return text
    .replace(/\*\*(.*?)\*\*/g, '*$1*') // Fett: **text** -> *text*
    .replace(/`([^\`]+)`/g, '`$1`'); // Inline-Code anpassen
}

waClient.initialize();

```

5. Risiken, Limits & Best Practices

1. Meta-Sperrrisiko (ToS):

1. Die Nutzung von `whatsapp-web.js` ist inoffiziell. Verwende nach Möglichkeit keine geschäftskritische oder private Hauptnummer für die Bot-Sitzung.
2. Keine Massennachrichten oder Spamschleifen erzeugen.

2. Schleifen-Prävention (Loop Avoidance):

1. Filter 1: Absender-ID (`fromUser.id === GITTER_BOT_USER_ID`) verwerfen.
2. Filter 2: Content-Präfixe (`[WhatsApp]`) im Stream verwerfen.

3. Ausfallsicherheit:

1. Den Dienst auf dem Server via Prozessmanager verwalten:

Bash



```

npm install -g pm2
pm2 start bridge.js --name "gitter-wa-bridge"
pm2 save
pm2 startup

```

Möchtest du das Setup direkt weiter vertiefen?

Docker-Setup mit PM2 für den Dauerbetrieb erstellen



Filterung für GitHub-Events im Gitter-Stream einbauen



WhatsApp-Steuerbefehle wie !mute oder !status implementieren

